TASK 9 – INTRODUCE ABSTRACTION

1. Introduction

Abstraction is an important object-oriented programming principle that hides implementation details and exposes only the essential features of an object. It helps separate what an object must do from how the operation is performed.

In the Campus Smart-Parking System, different vehicle types share common properties and operations, but their parking rates and additional capabilities may differ. The system also supports different user payment behaviours. Abstract classes and interfaces are therefore used to define common contracts while allowing individual classes to provide their own implementations.

2. Use of an Abstract Class

The `Vehicle` class is designed as an abstract class because it represents a general vehicle and should not be instantiated directly. Every vehicle has common information such as its registration number, but the parking rate depends on its specific type.

The `Vehicle` abstract class contains:

- A private registration number field to store the vehicle's unique identifier.
- A protected constructor to initialise the registration number.
- A public getter to access the registration number.
- An abstract method `ratePerHour()` that requires each concrete vehicle subclass to provide its hourly parking rate.
- A concrete method `calculateFee()` that calculates the parking fee using the vehicle's hourly rate.

The `ratePerHour()` method is declared abstract because the general Vehicle class cannot determine the rate for every vehicle category.

For example:

- `TwoWheeler` provides an hourly rate of Rs. 10.
- `Car` provides an hourly rate of Rs. 30.
- `ElectricCar` inherits the parking rate from `Car` and additionally supports charging.

This design ensures that every concrete vehicle class provides the required parking-rate implementation.

3. Use of an Interface

An interface is used to represent a capability that can be implemented by different classes, even when those classes do not belong to the same inheritance hierarchy.

The `Chargeable` interface represents the ability of a vehicle to use an EV charging facility. It declares the method:

```
double chargingFee(int kWh);
```

The `ElectricCar` class implements `Chargeable` because it supports battery charging. Ordinary cars and two-wheelers do not need to implement this interface.

A second interface, `Payable`, can represent the capability of making a payment. A visitor who pays the parking fee can implement this interface. The interface can define a `pay()` method that must be implemented by the visitor class.

Thus, interfaces represent optional capabilities rather than forcing every class to inherit behaviour that it does not need.

4. Why an Abstract Class and an Interface Are Used

An abstract class and an interface serve different purposes in this system.

Abstract Class – Vehicle	**Interface – Chargeable**
Represents a common vehicle identity.	Represents an additional charging capability.
Can contain instance variables such as registration number.	Defines behaviour that implementing classes must provide.
Can have constructors to initialise common state.	Cannot have constructors or ordinary per-object instance fields.
Can contain both abstract and concrete methods.	Can contain abstract methods and, in modern Java, default and static methods.
A class can extend only one class.	A class can implement multiple interfaces.

The `Vehicle` class is appropriate as an abstract class because all vehicles share state and common implementation. `Chargeable` is appropriate as an interface because charging is an optional capability that only some vehicles support.

5. Multiple Behaviours Through Interfaces

Java does not support multiple inheritance of classes. However, a class can implement multiple interfaces.

For example, an `ElectricCar` can extend the `Car` class and implement both `Chargeable` and another suitable capability interface.

Conceptually:

```
ElectricCar extends Car implements Chargeable, Payable
```

This allows the class to inherit common vehicle information and behaviour from `Car` while obtaining additional capabilities through interfaces.

The approach avoids creating unnecessary subclasses for every possible combination of vehicle features.

6. Default and Static Methods in Interfaces

Java interfaces can contain default and static methods.

Default method: A default method provides a common implementation that implementing classes can use without overriding it.

For example, the `Chargeable` interface may contain a default method called `chargingDescription()` that returns a general message indicating that the vehicle supports charging.

Static method: A static method belongs to the interface itself and can be called using the interface name.

For example, `Chargeable.isValidEnergy(kWh)` can validate whether the supplied charging energy is positive.

Default methods are useful when common behaviour must be shared by implementing classes. Static methods are useful for utility operations related to the interface.

7. Working of the Abstraction-Based Design

The abstraction-based design follows these steps:

1. The system creates an object of a concrete vehicle class such as `TwoWheeler`, `Car` or `ElectricCar`.
2. Each vehicle object obtains common registration information from the abstract `Vehicle` class.
3. The concrete vehicle class provides its own implementation of `ratePerHour()`.
4. The `calculateFee()` method uses the implemented hourly rate to calculate the parking fee.
5. If the vehicle supports charging, the system can access the `Chargeable` capability and calculate the additional charging fee.
6. A payment-related class can implement `Payable` to provide its own payment behaviour.

The abstract class defines the common vehicle structure, while interfaces define additional capabilities.

8. Alternative Concept and Complexity Analysis

The original design uses `Vehicle` as an abstract class and `Chargeable` as an interface. Consider reversing these roles by making `Vehicle` an interface with default methods and changing `Chargeable` into an abstract class.

8.1 Alternative Design

In the alternative design, `Vehicle` would become an interface containing methods such as `getRegNo()` and `ratePerHour()`. A default method could provide a common fee-calculation implementation.

The `Chargeable` type would become an abstract class containing charging-related state and methods. `ElectricCar` could extend `Chargeable` and implement `Vehicle`.

However, this creates limitations:

- An interface cannot have constructors, so it cannot initialise a registration number through a constructor.
- An interface cannot hold ordinary per-object instance state. Fields declared in an interface are implicitly public, static and final.
- A class can extend only one class. If `ElectricCar` extends `Chargeable`, it cannot also extend `Car`.
- The alternative structure makes sharing common vehicle state and charging behaviour less straightforward.

- Default methods provide behaviour but do not replace the constructor and instance-state capabilities of an abstract class.

8.2 Comparison of the Two Designs

Aspect	Original Design	Alternative Design
Vehicle representation	Abstract class	Interface
Charging representation	Interface	Abstract class
Constructors	Supported in `Vehicle`	Not supported in `Vehicle` interface
Common instance state	Stored in `Vehicle`	Must be stored in implementing classes or another class
Multiple behaviour support	Through interfaces	Through interfaces, but `Chargeable` as a class restricts class inheritance
Suitability	Suitable for shared vehicle identity and state	Less suitable when vehicles need common state and constructors

8.3 Time Complexity

The original design's `calculateFee()` method performs a constant number of arithmetic operations and one call to `ratePerHour()`. Therefore, its time complexity is O(1).

The alternative design can also calculate the fee in O(1) when its default method and implementation perform a fixed number of operations.

The time required for a charging-fee calculation is also O(1) in both designs, assuming the calculation uses a fixed number of arithmetic operations.

There is no inherent asymptotic performance advantage in changing an abstract class into an interface. The main difference is in the structure, state management and inheritance restrictions.

8.4 Space Complexity

In the original design, each vehicle object stores its registration number and any other instance data. The interface itself does not add ordinary per-object instance fields. The per-object space for a fixed set of fields is O(1).

In the alternative design, the vehicle implementation classes must store their own registration numbers because an interface cannot hold ordinary instance state. The `Chargeable` abstract class may also introduce charging-related instance fields.

The per-object space remains O(1) for a fixed number of fields, although the exact memory usage depends on the fields stored by each implementation.

9. Advantages of the Proposed Design

- **Code reusability:** Common vehicle properties and fee-calculation behaviour are maintained in one abstract class.
- **Extensibility:** New vehicle categories can extend `Vehicle` and provide their own parking rates.

- **Flexibility:** Interfaces allow optional capabilities such as charging and payment.
- **Reduced coupling:** The system depends on common abstractions rather than requiring every operation to know the concrete vehicle type.
- **Maintainability:** Changes to common vehicle behaviour can be made in the abstract class without duplicating the same logic in every subclass.

10. Conclusion

The Campus Smart-Parking System uses abstraction through an abstract `Vehicle` class and interfaces such as `Chargeable` and `Payable`. The abstract class is suitable for representing common vehicle state and behaviour, while interfaces represent additional capabilities that can be implemented independently.

Reversing the two concepts is possible in some designs, but it introduces limitations related to constructors, instance state and Java's single class inheritance. Both approaches can provide constant-time fee calculations, but the original design offers a more suitable structure for the parking system.

Therefore, the use of an abstract class for `Vehicle` and interfaces for optional capabilities is a justified and maintainable design choice.